ASIC Circuit Netlist Recognition Using Graph Neural Network

Xuenong Hong, Tong Lin, Yiqiong Shi, Bah Hwee Gwee
School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore
Email : {xuenong001,lintong, yqshi, ebhgwee}@ntu.edu.sg

2021 IEEE International Symposium on the Physical and Failure Analysis of Integrated Circuits (IPFA) | 978-1-6654-3988-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/IPFA53173.2021.9617311

***Abstract*—Circuit netlist recognition is an important step in the analysis flow of hardware assurance. The main objective in circuit netlist recognition is to classify circuit netlists into different categories according to their structural differences for the purposes of design verification and trojan detection. Traditional methods are usually based on human knowledge and statistical heuristics, which incur a large degree of error and cannot easily handle large ASIC circuit netlists. In this paper, we propose a machine learning solution based on graph neural network for automated circuit classification. By performing experiments on example synthesized ASIC circuits, we showed that our machine learning model produces highly accurate predictions of 98.3% accuracy.**

Keywords—hardware assurance, netlist recognition, graph neural network, machine learning

I. INTRODUCTION

Hardware assurance and security play an important role in semiconductor industry. Hardware assurance performs verifications in manufactured integrated circuits (ICs) to identify malicious modifications, known as hardware Trojans, for the prevention of possible Intellectual Property infringements and security attacks[1]-[3]. As the scale of circuit design increases dramatically, manual hardware verification becomes more and more challenging. Computer-aided hardware analysis aims to perform automated verification of manufactured circuits with their original design by inspections. Specifically, a manufactured IC is unpackaged and delayered, followed by imaging to obtain the high-resolution delayered IC images [4]. The circuit connections in the images are then extracted by computer-aided methods to recover the netlist of the circuit [5][6]. Subsequently, the circuit netlist needs to undergo analysis to verify the design at a functional and/or structural level, which is the most important step in hardware assurance analysis flow. Traditionally, this was done by heuristics-based[7][8] methods, library-based matching methods[9][10] with a cell library, or knowledge-based[11][12] embedding methods, which requires either human experience or hard to be generalized to complex designs.

Machine learning has shown astounding performance in many fields and can accommodate a large amount of data. Recent approaches employ machine learning for circuit classification. In [13], a CNN based approach is used for arithmetic circuit classification based on IC image analysis. However, the structural information of circuits is not captured by CNN framework. Graph-based analysis methods become increasingly popular in netlist analysis research fields due to the simplicity of representing circuits in graph models. Graph neural network (GNN) emerges to be a powerful framework for graph structural analysis and show promising results in many graph analysis tasks[14][15]. Many works employ GNNs for graph level classification. Similar ideas have been studied and generalized to circuit classification. Both [16][17] employs graph neural network for analog circuit classification with a graph pooling strategy. However, analog circuits are usually small designs and circuit understanding is less challenging as compared to large VLSI circuits. To the best of the author's knowledge, no prior work has been done on ASIC netlist using graph machine learning approaches.

In this work, we propose a machine learning solution for ASIC netlist recognition based on graph neural network. Our results show that the GNN model can successfully recognize circuits with different architectures by performing graph convolution on circuit graphs.

II. BACKGROUND

A. Netlist Recognition

The most challenging task in hardware analysis flow is to perform netlist recognition, which requires identifying circuit designs from a flatten netlist consists of millions of logical elements to obtain abstractions of each functional module in the IC. Specifically, given a circuit netlist recovered from a physical circuit, the objective is to extract the functional modules in the netlist, and subsequently determine the design of the given netlist for verification purposes. Since the extraction process usually depends on classifications of given netlists, we can reformulate the netlist recognition task as a classification problem, that is, we recognize a given circuit netlist by classifying it into predefined categories.

B. Graph Neural Network

Our machine learning model is mainly based on graph neural network for graph level embedding learning. A graph neural network is a machine learning model that can be directly applied to graph-structured data. For a graph $G\sim(V,E)$, graph neural network model is to learn a function that takes the inputs: A, the graph adjacency matrix, and X, the node level features and produce the output Z, the node level embedding for the graph. Mathematically, a multi-layer GNN with L layers can be rewritten as:

$$H^l = f_\theta(A, H^{l-1}) \tag{1}$$

with $H^0 = X$ and $H^L = Z$, H^{l-1} and H^l are the input features and output features at the l^{th} layer, respectively. The objective is to formulate f_θ as a parameterized function trainable via gradient descent.

Though the first GNN proposed a recurrent structure for learning graph recursively with neighborhood aggregation [18] until convergence, most recent works are based on graph convolutional network (GCN), where a few layers of graph convolution operations produce effectively good node level embedding. GCN performs spectral or spatial convolution for node embedding learning and is shown to have good discriminative power of different graph structures. Spatial graph convolution generalizes from the convolution in image domain to higher dimensional graph-

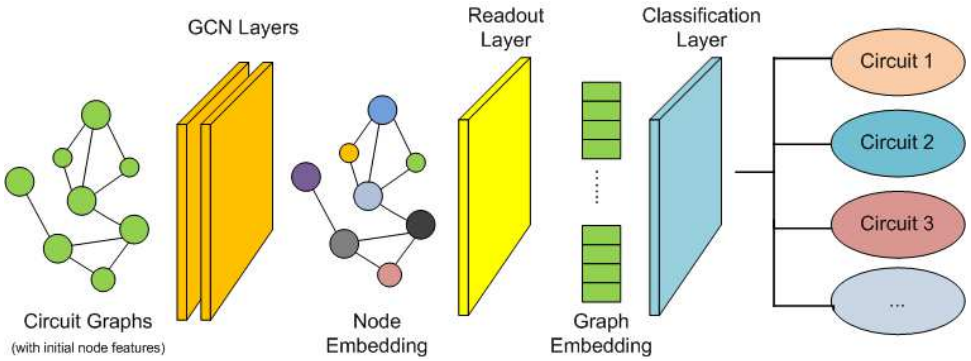

Fig 1: Architecture of GNN model for circuit classification

structured data, whilst spectral convolution derived from spectral graph theory which has a strong mathematical foundation.

Derived from spectral graph theory, the vanilla GCN is a spectral graph convolution[19] which defines a graph convolution operator g_θ on normalized adjacency matrix, where:

$$H^l = g_\theta(A, H^{l-1}) \approx \theta\left(D^{-\frac{1}{2}}AD^{\frac{1}{2}}\right)H^{l-1} \qquad (2)$$

with $D^{-\frac{1}{2}}AD^{\frac{1}{2}}$ being the normalized graph adjacency and θ as a single trainable parameter matrix for graph convolution operation, which can be trained by supervised loss or unsupervised loss or a combination of both. The output at the l^{th} GCN layer represents the learnt node level embedding of the graph. Further graph "readout" operation can be performed to obtain graph level representation, which is commonly done by averaging node features.

III. Proposed Methodology

In this section, we present our proposed machine learning model and training strategies used in our work.

GNN performs graph isomorphism by graph structural similarity. In circuit analysis, circuits can be classified according to their structural and functional similarity. It is also discussed that there exists a close relationship between circuits' structural similarity and functional similarity [20]. Therefore, we proposed to use a GNN based machine learning model for circuit netlist classification to classify circuits according to their structural similarity.

The GNN network used in our experiments consists of 2 shallow GCN layers for node embedding learning, one graph readout layer for graph level embedding, and one FCN layer with softmax operation for classification output. We adopted the GCN proposed in [19] in our model as it is a simple yet powerful formulation. Each GCN layer performs one graph convolution operation on the input graphs. Note that each graph convolution effectively performed neighborhood information exchange in a one-hop neighborhood. The architecture of the model used in our experiments is depicted in Fig.1. Circuit netlists are transformed into circuit graphs by representing every logic gate as a node in the graph, and their inter-connections as edges in the graph. We encode the logic gate type in one-hot encoding and use it as node features. Model input dimension is set to be the number of feature dimensions. We used a hidden dimension of 256 and the output dimension equals to the number of circuit netlist classes. By performing graph convolution, the GCN model embeds node features and graph connections into a 256-dimensional hidden representation and use it for classification. The model is trained in a supervised manner by a cross-entropy loss between the predicted result and its ground-truth label. By minimizing the cross-entropy loss, the model learns to embed graphs with similar structures to a similar embedding and enlarge the embedding distance between graphs with different structures.

Training data is key to the success of machine learning algorithms. However, data acquisition is extremely challenging in circuit domain due to the low variation of circuits with the same design. To have sufficient training data, we propose to train the machine learning model using generated circuits with different parameters (such as bit-width) for the same circuit design, since modifying the parameters of circuits will change the circuit structure while preserving the key architecture of the circuit designs. Therefore, by using circuits with different parameters, it effectively encourages the machine learning model to capture the key architecture differences among different design whilst ignoring other structural variations. We synthesize all circuits' Verilog into ASIC netlists using an open-source EDA tool called Qflow. We generate the circuit graph representation for each netlist by representing each gate as a node and the connections as edges in the circuit graphs.

IV. Experiment Results

To demonstrate that the GNN model can successfully classify ASIC circuits based on circuit graphs, we performed a case study on four designs of adder circuits with different architectures. The designs of adders are: 1) Ripple-Carry Adder (RCA); 2) Carry-Look-Ahead adder (CLA); 3) Carry-Select Adder (CSLA); and 4) Carry-Skip Adder (CSKA). The architectures of the four types of adders are depicted in Fig. 2. They all contain single or double chains of full adders (FAs) with extra logic connections. From their differences in architectures, we can expect that their netlist circuit graphs have different structural characteristics.

To have enough training and testing data, we generated adder circuits Verilogs with different bit-width from 8 bits to 64 bits using generate Verilog statement. We synthesize all circuits' Verilog into ASIC netlists and generate the circuit graph representation for each netlist by representing each gate as a node and the connections as edges in the circuit graphs. We encode gate types in a one-hot encoding

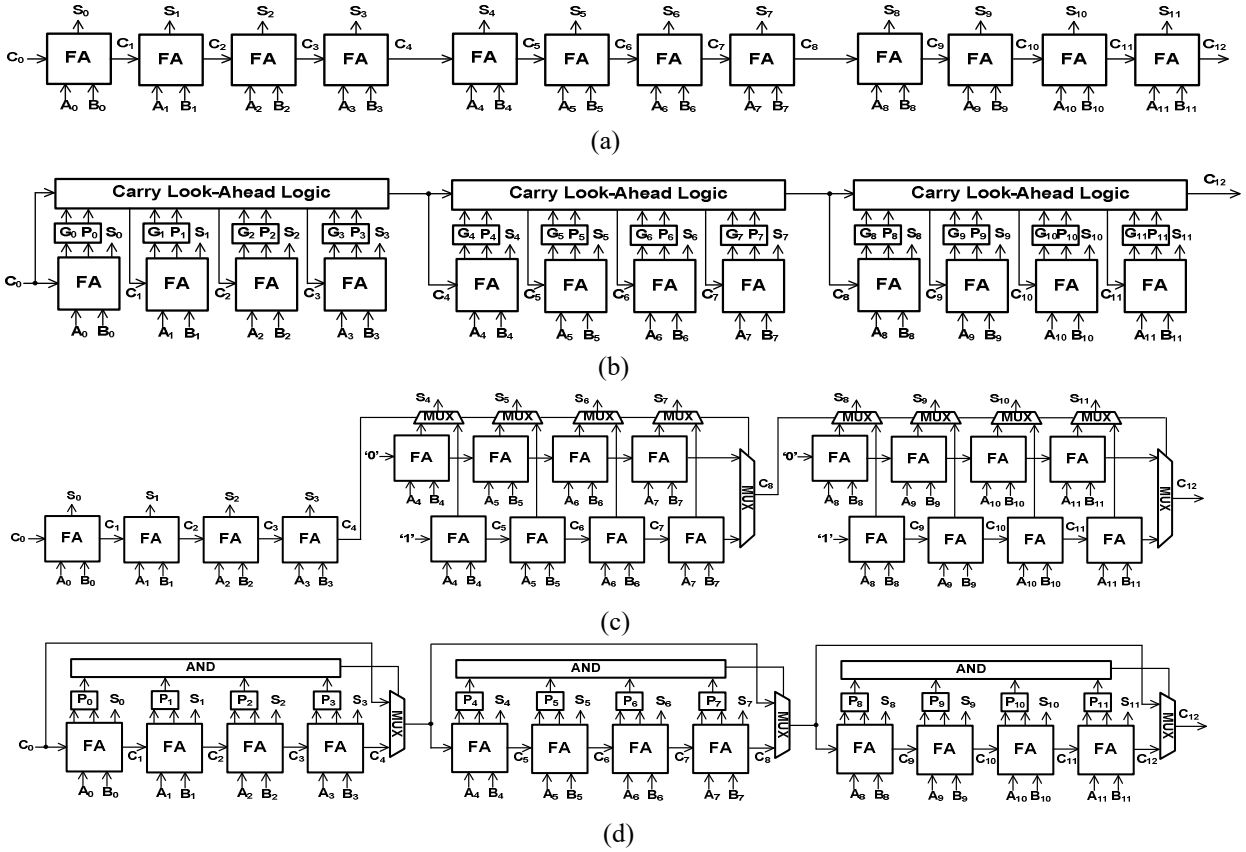

Fig 2: Architectures of adder circuits: the schematics of (a) RCA, (b) CLA, (c) CLSA, (d) CSKA. The labels in the schematics are as following: FA: full adder; S: sum; C: carry; G generate; P: propagate.

and use them as initial node features for the circuit graphs. The visualization of different adders' circuit graphs is depicted in Fig. 3, with each column showing one type of adder. Graphs are high dimensional data and are difficult to visualize. To visualize the graph, we embedded the graph into a 2-dimensional space by Kamada-Kawai layout [21]. From the 2-D embedded graph visualization, we can observe that the same types of adders, albeit having different bit-width, tend to have similar circuit graph structures, whilst different adders have different graph structures. Therefore, GNN can be employed to capture their structural differences in circuit graphs. As shown in Fig 3, circuits graphs of RCA and CLA are aligned with their architectures. For RCA, it is a single-chain structure whilst for CLA it is an expanded-single chain due to the additional 'carry-look-ahead' logics. Note that although CSLA and

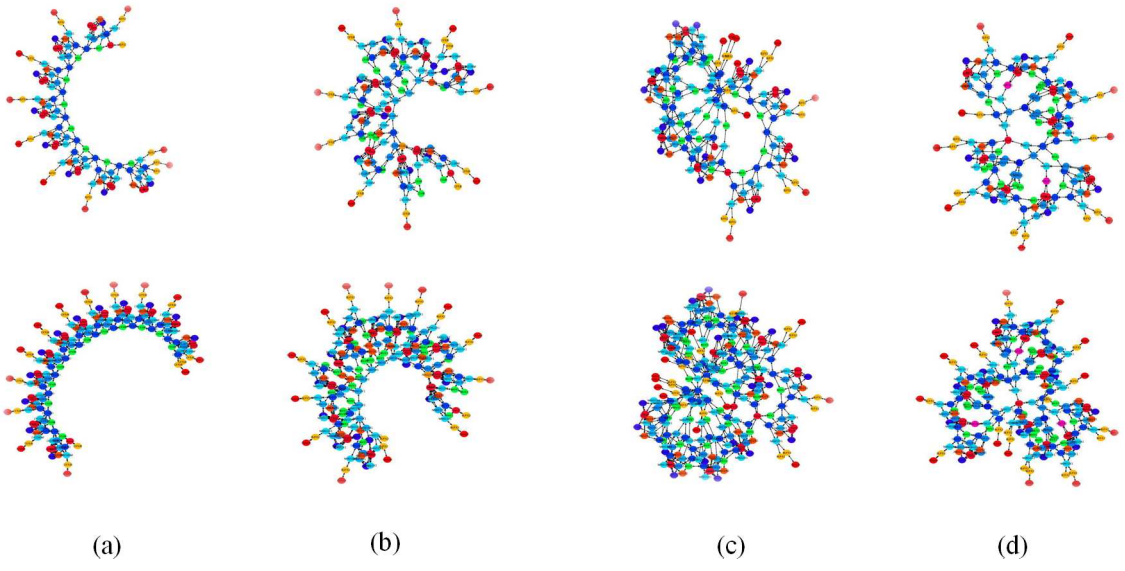

Fig 3: Visualization of Circuit Graphs: (a) 8-bits (top) and 16-bits (bottom) RCA; (b) 8-bits (top) and 16-bits (bottom) CLA; (c) 8-bits (top) and 16-bits (bottom) CSLA; (d) 8-bits (top) and 16-bits (bottom) CSKA. Colors of the nodes indicates the gate types of nodes, which we encode as node features.

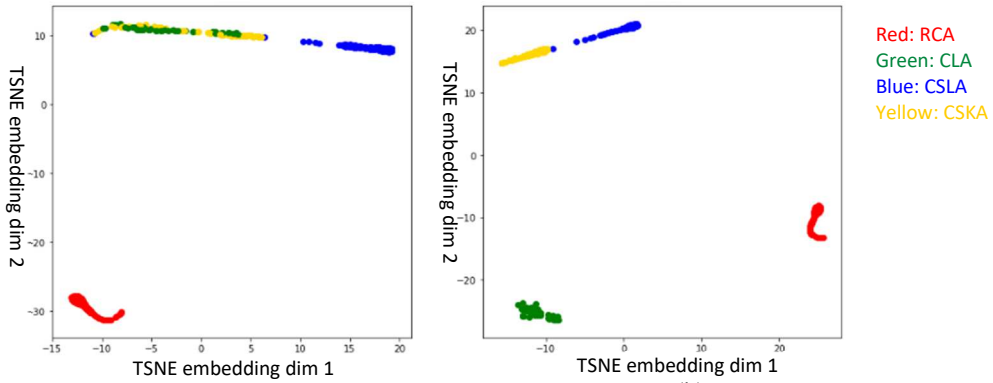

Fig 4: TSNE Visualization of graph embeddings after: (a) first GCN layer, (b) second GCN layer

CSKA have full adder chains in their architectures, the additional "select" logics and "skip" logics connects the "head" and "tail" of the full adder chains, resulting in an overall circular structure in the circuit graphs.

We randomly selected 20 adder circuits of each type with different bit-widths to form the train set and use the remaining circuits as the test set, resulting in a total of 80 circuits in the train set and 144 circuits in test set. We further verified that the circuit graphs in the test set and the train set are different and independent by looking at the average differences in the sizes of graphs in the train set and the test set. The model is trained with Adam optimizer at a learning rate of 10^{-3} for 100 epochs. It is then used to perform inference on circuits in the test set.

Table I presents the classification results on adder circuits using our GNN model. It perfectly distinguishes single-chain-structured RCA (Fig. 3(a)) and expanded-single-chain-structured CLA (Fig. 3(b)) from the rest of the circuits due to the simplicity in the graph structure. Although humans fail to observe highly differentiable structures in CSLA and CSKA, the GNN model successfully captures the graph structural differences, albeit with a lower confidence level as compared to RCA and CLA.

To further study the discriminative power of GCN layers, we also visualized the graph embedding after each GCN layer using TSNE [22] in Fig.4. It clearly shows that the graph embedding of RCA is separated from the rest after the first GCN. This is mainly because RCA has the most distinguishable structures as compared to the other three circuits. The embeddings of CSLA are also apart from the others, however with smaller distance, showing that these

Table I: Classification Result on Adder Circuits Netlist

Circuits	Recall	Precision	F1-Score
RCA	1.000	1.000	1.000
CLA	1.000	1.000	1.000
CSLA	0.933	1.000	0.965
CSKA	1.000	0.938	0.968

circuits are more similar to other types of circuits. It is mentionable that the embedding of CLA and CSKA are not separable after one layer of GCN, due to GCN failed to differentiate their strong architectural similarity within one-hop of graph convolution. Note that although their graph visualizations look very different, they, however, this visual difference is only caused by additional common connections to 'gnd' (indicated as '0' in Fig. 2) in CSLA, resulting in many small "star-graph[1]" structures (as in Fig. 3(c)). Therefore, although having very different appearances in visualization, their graphs are still structurally similar if we consider the graph edit distance [23]. After the second GCN layer, the graph embeddings of CLAA and CSLA are also clearly separated by another graph convolution operation. This gives the final embeddings and leads to the predictions of the classes of circuits. The model achieves an overall accuracy of 98.3%.

V. Future Work

For complex circuit designs, such as digital IC cores and other logic circuits with multiple design hierarchies, directly applying GNN may not yield good prediction outcomes. Further experiments can be performed to study methods for applying GNN to giant circuits using strategies such as graph pooling/clustering for circuit hierarchical classification. Furthermore, memory issue has been a major concern for applying GNN to large graphs. Strategies for overcoming memory limitations can be investigated.

The discriminative power of GNN largely depends on node features learning. More differentiable node features highly encourage the learning of GNN models. Future direction can be investigating different structural and/or functional features associated with gates/cells in the circuit netlist to strengthen GNN models and generalize to classifications of bigger and more complex designs.

VI. Conclusion

In this paper, we developed a GNN framework for ASIC circuit netlist recognition. We showed that our GNN model archives good performance in circuit netlist recognition task. We performed a case study on adder circuits to demonstrate that GNN can successfully distinguish adder circuit graphs within two hops of graph convolution. The result demonstrates that GNN can perform circuit logics discrimination by analysing circuit structure.

[1]A star-graph is a tree on n nodes with one node having $n-1$ degree and the other $n-1$ nodes having degree 1.

REFERENCES

[1] D. Mukhopadhyay and R. S. Chakraborty, *Hardware security: Design, threats, and safeguards*. 2014, pp. 1-535.

[2] M. Rostami, F. Koushanfar, and R. Karri, "A Primer on Hardware Security: Models, Methods, and Metrics," *Proceedings of the IEEE,* vol. 102, no. 8, pp. 1283-1295, 2014, doi: 10.1109/JPROC.2014.2335155.

[3] M. Rahman and N. Asadizanjani, "Failure Analysis For Hardware Assurance and Security," *Electronic Device Failure Analysis,* vol. 21, p. 16, 07/15 2019.

[4] R. Torrance and D. James, "The State-of-the-Art in IC Reverse Engineering," 2009, pp. 363-381.

[5] D. Cheng, Y. Shi, T. Lin, B. Gwee, and K. Toh, "Hybrid ${K}$ -Means Clustering and Support Vector Machine Method for via and Metal Line Detections in Delayered IC Images," *IEEE Transactions on Circuits and Systems II: Express Briefs,* vol. 65, no. 12, pp. 1849-1853, 2018, doi: 10.1109/TCSII.2018.2827044.

[6] X. Hong, D. Cheng, Y. Shi, T. Lin, and B. H. Gwee, "Deep Learning for Automatic IC Image Analysis," in *2018 IEEE 23rd International Conference on Digital Signal Processing (DSP)*, 19-21 Nov. 2018 2018, pp. 1-5, doi: 10.1109/ICDSP.2018.8631555.

[7] Y. Shi, C. W. Ting, B.-H. Gwee, and Y. Ren, "A highly efficient method for extracting FSMs from flattened gate-level netlist," 2010. [Online]. Available: https://doi.org/10.1109/ISCAS.2010.5537093.

[8] Y. Shi, B.-H. Gwee, Y. Ren, T. K. Phone, and C. W. Ting, "Extracting functional modules from flattened gate-level netlist," 2012. [Online]. Available: https://doi.org/10.1109/ISCIT.2012.6380958.

[9] M. Meissner and L. Hedrich, "FEATS: Framework for Explorative Analog Topology Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems,* vol. 34, no. 2, pp. 213-226, 2015, doi: 10.1109/TCAD.2014.2376987.

[10] T. Massier, H. Graeb, and U. Schlichtmann, "The Sizing Rules Method for CMOS and Bipolar Analog Integrated Circuit Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems,* vol. 27, no. 12, pp. 2209-2222, 2008, doi: 10.1109/TCAD.2008.2006143.

[11] R. Harjani, R. A. Rutenbar, and L. R. Carley, "A Prototype Framework for Knowledge-Based Analog Circuit Synthesis," in *24th ACM/IEEE Design Automation Conference*, 28 June-1 July 1987 1987, pp. 42-49, doi: 10.1145/37888.37894.

[12] P. Wu, M. P. Lin, T. Chen, C. Yeh, X. Li, and T. Ho, "A Novel Analog Physical Synthesis Methodology Integrating Existent Design Expertise," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems,* vol. 34, no. 2, pp. 199-212, 2015, doi: 10.1109/TCAD.2014.2379630.

[13] L. M. Silva, F. V. Andrade, A. O. Fernandes, and L. F. M. Vieira, "Arithmetic Circuit Classification Using Convolutional Neural Networks," in *2018 International Joint Conference on Neural Networks (IJCNN)*, 8-13 July 2018 2018, pp. 1-7, doi: 10.1109/IJCNN.2018.8489382.

[14] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A Comprehensive Survey on Graph Neural Networks," *IEEE Transactions on Neural Networks and Learning Systems,* vol. 32, no. 1, pp. 4-24, 2021, doi: 10.1109/TNNLS.2020.2978386.

[15] J. Zhou *et al.*, "Graph Neural Networks: A Review of Methods and Applications," p. arXiv:1812.08434. [Online]. Available: https://ui.adsabs.harvard.edu/abs/2018arXiv181208434Z

[16] K. Kunal *et al.*, *GANA: Graph Convolutional Network Based Automated Netlist Annotation for Analog Circuits*. 2020, pp. 55-60.

[17] Y. Li *et al.*, "A Customized Graph Neural Network Model for Guiding Analog IC Placement," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2-5 Nov. 2020 2020, pp. 1-9.

[18] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The Graph Neural Network Model," *IEEE Transactions on Neural Networks,* vol. 20, no. 1, pp. 61-80, 2009, doi: 10.1109/TNN.2008.2005605.

[19] T. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," 09/09 2016.

[20] J. Baehr, A. Bernardini, G. Sigl, and U. Schlichtmann, "Machine learning and structural characteristics for reverse engineering," *Integration,* vol. 72, pp. 1-12, 2020/05/01/ 2020, doi: https://doi.org/10.1016/j.vlsi.2019.10.002.

[21] T. Kamada and S. Kawai, "An algorithm for drawing general undirected graphs," *Information Processing Letters,* vol. 31, no. 1, pp. 7-15, 1989/04/12/ 1989, doi: https://doi.org/10.1016/0020-0190(89)90102-6.

[22] L. van der Maaten and G. Hinton, "Viualizing data using t-SNE," *Journal of Machine Learning Research,* vol. 9, pp. 2579-2605, 11/01 2008.

[23] A. Sanfeliu and K. Fu, "A distance measure between attributed relational graphs for pattern recognition," in *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 3, pp. 353-362, May-June 1983, doi: 10.1109/TSMC.1983.6313167.